GUIA DE ESTUDO

Desafio Backend Go Processamento Distribuído de Apostas

O que foi construído em cada spec, por que foi feito assim e quais conceitos de Go aparecem, para quem vem de C#, Python e TypeScript.

Repositório: github.com/rafaelcorreadev1/junglegaming
Atualizado até a spec 05 · 13/10/2026

Como usar este guia

Cada spec concluída e testada vira um capítulo. Todos os capítulos seguem a mesma ordem:

1. **O que a spec pedia** — o requisito do desafio.
2. **O que foi construído** — os arquivos e o papel de cada um.
3. **Como funciona** — o fluxo, passo a passo.
4. **Conceitos de Go** — o que é novo, comparado com C# e TypeScript.
5. **Decisões** — as escolhas feitas e o motivo.
6. **Como foi verificado** — o que foi executado e o resultado.
7. **Perguntas de revisão** — para testar se você entendeu.

Leia com o código aberto ao lado. Os trechos aqui são recortes; o arquivo completo está no caminho indicado.

Progresso

#	SPEC	STATUS
01	Bootstrap: módulo Go, Fx, config, Docker Compose	Concluída
02	Money	Concluída
03	Domínio: Wallet, Ledger, WagerTransaction	Concluída
04	Schema e migrations	Concluída
05	Persistência e transação SQL	Concluída
06	Carteira: abertura, leitura, ledger paginado	A fazer
07	Operação: BET / WIN / LOSS e idempotência	A fazer
08	Reversões e referências pendentes	A fazer
09	Autenticação e autorização	A fazer
10	Outbox e eventos	A fazer
11	Consumidor SQS, inbox e DLQ	A fazer
12	Reconciliação	A fazer
13	Observabilidade	A fazer
14	Ciclo de vida Fx e shutdown	A fazer
15	Testes de integração, concorrência e recuperação	A fazer
16	Documentação e entrega	A fazer

O desafio em um parágrafo

Um serviço que movimenta carteiras de jogadores a partir de operações enviadas por provedores de jogos: aposta (`BET`), ganho (`WIN`), perda (`LOSS`), devolução (`REFUND`) e estorno (`ROLLBACK`). As operações chegam por HTTP e por fila (SQS), e o resultado financeiro precisa ser o mesmo nos dois caminhos, mesmo com várias instâncias rodando e com falhas no meio do processamento.

Mapa mental: Go para quem vem de C# e TypeScript

EM GO	O EQUIVALENTE QUE VOCÊ CONHECE
`go.mod` / `go.sum`	`.csproj` / `package.json` + lockfile
Pasta `internal/`	`internal` do C#: o compilador impede import de fora do módulo
Nome com maiúscula	`public` ; minúscula é privado ao pacote
`struct` + função `NewX`	Classe + construtor
`interface` implícita	Interface estrutural do TypeScript: não existe `implements`
`error` como retorno	Exceção, só que retornada e checada com `if err != nil`
`context.Context`	`CancellationToken`
`defer`	`finally` / `using`
Goroutine (`go f()`)	`Task.Run`
Uber Fx	`IServiceCollection` / modules do NestJS

SPEC 01

Bootstrap: módulo Go, Fx, config e Docker Compose

Um processo Go que sobe, valida a configuração, conecta no Postgres e responde dois endpoints de saúde. Ainda não há nenhuma regra de negócio.

1. O que a spec pedia

- Versão do Go declarada em `go.mod` e no `Dockerfile`; dependências versionadas.
- Composição da aplicação com Uber Fx, com injeção por construtores.
- Configuração validada na inicialização: o processo não sobe com config inválida.
- Ambiente local com Docker Compose e um `.env.example` sem segredos.

2. O que foi construído

ARQUIVO	PAPEL
`cmd/app/main.go`	Ponto de entrada: carrega a config e entrega o controle ao Fx
`internal/config/config.go`	Lê e valida as variáveis de ambiente
`internal/app/app.go`	A composição: lista os módulos que formam a aplicação
`internal/infra/observability/logger.go`	Logger em JSON
`internal/infra/postgres/pool.go`	Pool de conexões com o Postgres
`internal/infra/httpapi/server.go`	Rotas e servidor HTTP
`internal/infra/httpapi/health.go`	Endpoints `/health/live` e `/health/ready`
`Dockerfile`, `docker-compose.yml`	Imagem da aplicação e ambiente local

3. Como funciona

1. `main` lê as variáveis de ambiente e monta a `Config`.
2. Config inválida: imprime o erro e sai com código 1. Nada mais é criado.
3. O Fx recebe os construtores e descobre a ordem: logger, pool, handler, router, servidor.
4. O Fx executa os `OnStart`: ping no banco e depois abertura da porta HTTP.
5. O processo fica parado esperando um sinal (`SIGTERM` ou Ctrl+C).
6. No sinal, o Fx executa os `OnStop` na ordem inversa: o servidor HTTP para e depois o pool fecha.

4. Conceitos de Go

Erro é valor de retorno

Go não tem exceção. A função que pode falhar devolve o erro como último retorno e quem chama é obrigado a decidir o que fazer com ele.

```
cfg, err := config.Load()
if err != nil {
    fmt.Fprintln(os.Stderr, err)
    os.Exit(1)
}
```

Embrulhar erro com `%w`

O erro original fica guardado dentro do novo, como a `InnerException` do C#. É o que permite, mais à frente, classificar erros com `errors.Is`, uma exigência do desafio.

```
if err := pool.Ping(ctx); err != nil {
    return fmt.Errorf("ping postgres: %w", err)
}
```

Visibilidade pela primeira letra

`Config`, `Load` e `NewPool` começam com maiúscula e são públicos. `valueOr` e `writeJSON` começam com minúscula e só existem dentro do próprio pacote.

Struct, construtor e método

Não há classes. Os dados ficam em uma `struct`, o construtor é uma função comum chamada `NewAlgo` por convenção, e o método é uma função com um *receiver*, que faz o papel do `this`.

```
type HealthHandler struct {
    pool   *pgxpool.Pool
    logger *slog.Logger
}

func NewHealthHandler(pool *pgxpool.Pool, logger *slog.Logger) *HealthHandler {
    return &HealthHandler{pool: pool, logger: logger}
}

func (h *HealthHandler) Live(w http.ResponseWriter, _ *http.Request) {
    writeJSON(w, http.StatusOK, map[string]string{"status": "ok"})
}
```

O `*` indica ponteiro (referência ao mesmo objeto) e o `&` cria esse ponteiro. O `_` significa "recebo o parâmetro, mas não uso".

`context.Context` e `defer`

O `context` vai sempre como primeiro parâmetro e carrega cancelamento e prazo. No readiness, ele limita o ping a 2 segundos. O `defer` agenda a linha para o fim da função, com ou sem erro.

```
ctx, cancel := context.WithTimeout(r.Context(), readinessTimeout)
defer cancel()

if err := h.pool.Ping(ctx); err != nil { ... }
```

Goroutine

`go func() { ... }()` inicia uma execução concorrente. O servidor HTTP roda em uma goroutine porque `Serve` bloqueia até o servidor fechar, e a inicialização precisa continuar.

5. Uber Fx

O Fx é o container de injeção de dependência. Você não registra "interface para implementação": entrega as funções construtoras e ele lê as assinaturas.

```
func NewRouter(health *HealthHandler) http.Handler
```

O Fx lê isso como: para criar um `http.Handler`, preciso de um `*HealthHandler`. Ele procura quem produz esse tipo e chama na ordem certa.

FUNÇÃO	O QUE FAZ
`fx.Provide(NewX)`	Registra um construtor. Só executa se alguém precisar do resultado.
`fx.Invoke(f)`	Executa sempre. É o que força a construção do servidor.
`fx.Supply(valor)`	Registra um valor pronto, sem construtor. Usado para a `Config`.
`fx.Module("nome", ...)`	Agrupa os registros de um pacote.
`fx.Lifecycle`	Onde cada componente registra `OnStart` e `OnStop`.

A composição inteira, em `internal/app/app.go`:

```
func Options(cfg config.Config) fx.Option {
    return fx.Options(
        fx.Supply(cfg),
        fx.StartTimeout(cfg.ShutdownTimeout),
        fx.StopTimeout(cfg.ShutdownTimeout),
        fx.WithLogger(...),
        observability.Module,
        postgres.Module,
        httpapi.Module,
    )
}
```

E o ciclo de vida do pool, em `internal/infra/postgres/pool.go`:

```
lc.Append(fx.Hook{
    OnStart: func(ctx context.Context) error {
        if err := pool.Ping(ctx); err != nil {
            return fmt.Errorf("ping postgres: %w", err)
        }
        return nil
    },
    OnStop: func(context.Context) error {
        pool.Close()
        return nil
    },
})
```

Por que a ordem de parada sai certa sozinha. O servidor depende do handler, que depende do pool. Então o pool é construído primeiro e registra seu hook primeiro. O Fx executa os `OnStop` na ordem inversa dos `OnStart`: o servidor para de receber requisições antes de o pool fechar.

6. Decisões

DECISÃO	MOTIVO
A config devolve todos os erros de uma vez (`errors.Join`)	Quem configura o ambiente corrige tudo em uma tentativa
A config recebe uma função `getenv` em vez de chamar `os.Getenv`	O teste passa um mapa e não altera o ambiente real
A config é carregada no `main`, antes do Fx	Os timeouts do próprio Fx vêm dela
A porta é aberta no `OnStart`, fora da goroutine	Porta ocupada vira falha de inicialização, em vez de erro perdido em background
O ping no banco fica no `OnStart`	Criar o pool não abre conexão; só o ping valida o banco
Composição em `internal/app`, fora do `main`	Permite testar o grafo de dependências sem subir nada
Dockerfile em dois estágios, com imagem final *distroless*	Go gera um binário único; a imagem final não precisa de compilador nem de shell

Liveness e readiness

ENDPOINT	PERGUNTA QUE RESPONDE	VERIFICA
`GET /health/live`	O processo está de pé?	Nada externo
`GET /health/ready`	Consigo atender requisições?	Postgres (o SQS entra na spec 10)

Se o banco cair, o orquestrador para de mandar tráfego (ready falha), mas não reinicia o processo (live continua ok), porque reiniciar não resolveria.

7. Testes

O arquivo de teste fica ao lado do código, com sufixo `_test.go`, e usa só o pacote `testing` da biblioteca padrão. O formato idiomático é o teste em tabela, equivalente ao `[Theory]` do xUnit:

```
tests := []struct {
    name    string
    env     map[string]string
    wantErr string
}{
    {name: "missing database url", env: map[string]string{}, wantErr: "DATABASE_URL is
required"},
    ...
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        _, err := FromEnv(envFrom(tt.env))
        ...
    })
}
```

`internal/app/app_test.go` usa `fx.ValidateApp`: confere que todo construtor tem as dependências de que precisa, sem iniciar nada.

8. Como foi verificado

VERIFICAÇÃO	RESULTADO
`gofmt -l .` e `go vet ./...`	Sem saída
`go test ./...`	Passou
`go test -race ./...` (em container Linux)	Passou
`docker compose up --build`	`/health/live` e `/health/ready` respondem 200
Postgres parado	`/health/ready` 503, `/health/live` 200
Sem `DATABASE_URL` e com `LOG_LEVEL` inválido	Os dois erros impressos, código de saída 1
`docker compose stop app`	Servidor para, pool fecha, código de saída 0

> **O `-race` não roda direto no Windows.** O detector de condição de corrida exige um compilador C, que não vem com o Go no Windows. A saída é executar em container:
>
> `docker run --rm -v "${PWD}:/src" -v wagering-gomod:/go/pkg/mod -w /src golang:1.27 go test -race ./...`

9. Comandos do dia a dia

COMANDO	PARA QUÊ
`go run ./cmd/app`	Roda a aplicação (precisa de `DATABASE_URL`)
`go test ./...`	Roda todos os testes; `./...` é "este diretório e todos abaixo"
`go vet ./...`	Análise estática
`gofmt -l .`	Lista arquivos fora do padrão de formatação
`go mod tidy`	Ajusta `go.mod` e `go.sum` ao que o código importa
`docker compose up --build`	Sobe Postgres e aplicação

10. Perguntas de revisão

Por que existe um `fx.Invoke(func(*http.Server) {})` com corpo vazio?

Porque o Fx só constrói o que alguém pede. Sem essa linha ninguém pediria o servidor, e ele nunca seria criado nem iniciado.

O que acontece se a porta 8080 já estiver em uso?

O `net.Listen` falha dentro do `OnStart`, o Fx aborta a inicialização e executa os `OnStop` do que já tinha iniciado (o pool fecha).

Por que o pool é fechado depois do servidor HTTP no shutdown?

Porque o servidor ainda pode estar terminando requisições que usam o banco. Fechar o pool antes quebraria essas requisições.

Qual a diferença entre `fx.Provide` e `fx.Supply`?

`Provide` registra uma função que cria o valor; `Supply` registra o valor já pronto.

Por que `FromEnv` recebe uma função em vez de ler o ambiente diretamente?

Para o teste controlar as variáveis sem alterar o ambiente do processo, o que também permite rodar testes em paralelo.

Com o banco fora do ar, por que `/health/live` continua 200?

Liveness diz se o processo precisa ser reiniciado. Banco fora não se resolve reiniciando a aplicação; quem sinaliza isso é o readiness.

Ordem de leitura do código

1. `cmd/app/main.go`
2. `internal/config/config.go` e o teste ao lado
3. `internal/app/app.go`
4. `internal/infra/postgres/pool.go`
5. `internal/infra/httpapi/server.go` e `health.go`

SPEC 02

Money

O tipo que representa dinheiro em todo o sistema: centavos em `int64` mais a moeda. Nenhum valor monetário passa por ponto flutuante.

1. O que a spec pedia

- Value object imutável com valor e moeda.
- Criação a partir de string decimal, zero por moeda, soma, subtração, negação, comparação e serialização.
- Proibido `float32` e `float64` em parsing, cálculo, serialização e persistência. É critério eliminatório.
- Escala fixa de duas casas e moeda ISO 4217.
- Rejeitar vazio, `NaN`, `Infinity`, notação científica e casas a mais, sem arredondar.
- Operações só entre valores da mesma moeda.
- Tratar overflow no parsing, na soma, na subtração e na negação.

2. Por que não usar float

Ponto flutuante guarda números em base 2, e a maioria das frações decimais não tem representação exata nela. Em qualquer linguagem, `0.1 + 0.2` resulta em `0.30000000000000004`. Em um sistema financeiro, esse resíduo vira centavo sobrando ou faltando depois de milhares de operações.

A saída é guardar a quantia como inteiro na menor unidade da moeda: `25.00` vira `2500` centavos. Inteiros somam e subtraem de forma exata.

3. O que foi construído

ARQUIVO	PAPEL
`internal/domain/money.go`	O tipo `Money`, seus erros e o parser
`internal/domain/money_test.go`	Testes em tabela; 100% das instruções cobertas

O pacote `domain` importa só a biblioteca padrão. Essa é a regra do desafio: o domínio não conhece Fx, HTTP, SQS nem banco.

A interface pública

FUNÇÃO OU MÉTODO	O QUE FAZ
`ParseMoney("25.00", "BRL")`	Cria a partir do contrato externo (string decimal)
`MoneyFromMinorUnits(2500, "BRL")`	Cria a partir de centavos; é o caminho de volta do banco
`ZeroMoney("BRL")`	Zero na moeda informada
`Add` , `Sub` , `Neg`	Devolvem um novo `Money` e um erro
`Cmp` , `Equal`	Comparação; `Cmp` devolve -1, 0 ou 1
`IsZero` , `IsPositive` , `IsNegative`	Perguntas que as regras de negócio vão usar
`Amount()` , `MinorUnits()` , `Currency()`	Leitura: `"25.00"` , `2500` , `"BRL"`
`MarshalJSON` , `UnmarshalJSON`	Conversão de e para `{"amount":"25.00","currency":"BRL"}`

4. Como funciona

Imutabilidade

```
type Money struct {
    minorUnits int64
    currency   string
}
```

Os dois campos começam com minúscula, então nenhum outro pacote consegue alterá-los. Os métodos usam receiver por valor (`func (m Money)` , sem `*`): cada chamada trabalha em uma cópia, e o resultado é sempre um `Money` novo. É o equivalente a um `readonly record struct` do C#.

O parser, passo a passo

1. Separa o sinal `-` , se houver. O sinal `+` não é aceito.
2. Divide no ponto: parte inteira e parte decimal.
3. Exige só dígitos de `0` a `9` nas duas partes. Isso elimina `NaN` , `Infinity` , `1e3` , espaços e vírgulas de uma vez.
4. Rejeita mais de duas casas decimais. Não há arredondamento.
5. Completa a parte decimal com zeros: `"25.5"` vira `"2550"` .
6. Converte dígito a dígito para `int64` , checando overflow antes de cada multiplicação.

Não se usa `strconv.ParseFloat` nem nenhuma função de conversão pronta: o parser é uma validação de caracteres seguida de aritmética de inteiros.

Overflow

Em Go, somar além do limite de um `int64` não gera erro: o número "dá a volta" e vira negativo, em silêncio. Por isso a checagem acontece **antes** da operação:

```
a, b := m.minorUnits, other.minorUnits
if (b > 0 && a > math.MaxInt64-b) || (b < 0 && a < math.MinInt64-b) {
    return Money{}, fmt.Errorf("%w: %s + %s", ErrMoneyOverflow, m, other)
}
return Money{minorUnits: a + b, currency: m.currency}, nil
```

A leitura é: "se `b` é positivo e `a` já está a menos de `b` do teto, a soma estouraria". A comparação é reescrita de forma que ela própria nunca estoure.

> **O caso da negação.** Um `int64` vai de -9.223.372.036.854.775.808 a +9.223.372.036.854.775.807: existe um negativo a mais. Negar o menor valor não tem resultado possível, então `Neg()` devolve erro nesse caso.

Erros classificáveis

```
var (
    ErrInvalidMoney     = errors.New("money is not initialized")
    ErrInvalidAmount    = errors.New("invalid money amount")
    ErrInvalidCurrency  = errors.New("invalid currency")
    ErrCurrencyMismatch = errors.New("currency mismatch")
    ErrMoneyOverflow    = errors.New("money amount overflow")
)
```

São os *erros sentinela*: variáveis públicas que servem de identidade do erro. Quem recebe o erro pergunta qual é com `errors.Is`, mesmo que ele tenha sido embrulhado com detalhes pelo `%w`:

```
_, err := brl.Add(usd)
if errors.Is(err, domain.ErrCurrencyMismatch) { ... }
```

Em C# isso seria um `catch (CurrencyMismatchException)`. Mais à frente, a camada HTTP usa essa classificação para decidir entre 400, 409 e 422.

O valor zero de uma struct

Em Go, toda variável nasce com um "valor zero": `var m Money` é válido para o compilador e contém `0` centavos e moeda vazia. Não existe `null` para structs. Como o desafio exige rejeitar valores não inicializados, a moeda vazia é o marcador: `IsValid()` devolve falso, e toda operação e a serialização devolvem `ErrInvalidMoney`.

JSON

O pacote `encoding/json` procura dois métodos no tipo. Se existirem, usa-os no lugar do comportamento padrão. É o equivalente a um `JsonConverter` do C#.

```
func (m Money) MarshalJSON() ([]byte, error)      // Money  -> JSON
func (m *Money) UnmarshalJSON(data []byte) error  // JSON   -> Money
```

`UnmarshalJSON` é o único método com receiver de ponteiro (`*Money`), porque precisa preencher a variável de quem chamou. Se o conteúdo for inválido, a variável não é alterada.

O campo `amount` é declarado como `string`. Se o cliente mandar um número JSON (`"amount": 25.00`, sem aspas), a decodificação falha.

5. Decisões

DECISÃO	MOTIVO
`int64` em centavos, sem biblioteca decimal	Sem dependência externa; mapeia direto para `BIGINT` no Postgres
Limite: cerca de 92 quatrilhões por valor	É o teto do `int64` dividido por 100; fica documentado como limitação
Aceita `"25"`, `"25.0"` e `"25.00"`; todas viram `"25.00"`	São o mesmo valor, sem perda. A forma normalizada é a que entra no hash de idempotência (spec 07)
Rejeita `"25.000"`, mesmo sendo o mesmo valor	O desafio pede rejeição de escala excedente; aceitar zeros extras abriria exceção à regra
Rejeita zeros à esquerda, `+`, `-0.00`, `.50` e `25.`	Uma única forma de escrever cada valor reduz ambiguidade
Só moedas ISO 4217 de duas casas	A escala é fixa em 2; `JPY`, sem centavos, ficaria errado
Moeda em minúsculas é rejeitada	Não normalizar em silêncio
`ParseMoney` aceita negativo	Diferenças negativas são serializadas (reconciliação). Proibir negativo em aposta e saldo é regra das entidades, na spec 03

6. Conceitos de Go

CONCEITO	ONDE APARECE	PARALELO
Receiver por valor ou por ponteiro	`(m Money)` nos métodos; `(m *Money)` em `UnmarshalJSON`	Passagem por cópia ou por referência (`ref`)
Valor zero	`Money{}` é inválido de propósito	`default(T)` do C#
Erro sentinela + `errors.Is`	`ErrInvalidAmount` e os demais	`catch` por tipo de exceção
Múltiplos retornos	`Add` devolve `(Money, error)`	Tupla, ou padrão `TryParse`
Struct tags	`` `json:"amount"` ``	`[JsonPropertyName("amount")]`
Conjunto com `map[string]struct{}`	Lista de moedas aceitas	`HashSet<string>`; `struct{}` ocupa zero bytes
`t.Helper()`	`mustMoney` nos testes	Faz a falha apontar para a linha do teste, não para o helper

7. Como foi verificado

VERIFICAÇÃO	RESULTADO
`go test ./internal/domain/ -cover`	Passou; 100% das instruções
`go test -race ./...` (em container)	Passou
`gofmt -l .` e `go vet ./...`	Sem saída
Busca por `float32` e `float64` nos arquivos `.go`	Nenhuma ocorrência

O que os testes cobrem

- **Aceitos:** 12 formas válidas, incluindo os dois extremos do `int64`.
- **Rejeitados:** 23 entradas inválidas, de `NaN` a dígitos não ASCII.
- **Overflow:** no parsing, na soma, na subtração e na negação, incluindo os casos de fronteira que *não* devem estourar.
- **Moedas:** soma, subtração e comparação entre `BRL` e `USD` devolvem `ErrCurrencyMismatch`.
- **Não inicializado:** `Money{}` é rejeitado em sete operações.
- **JSON:** ida e volta, e oito corpos inválidos.

8. Perguntas de revisão

Por que `Add` devolve erro, se somar dois inteiros nunca falha em Go?

Por dois motivos: as moedas podem ser diferentes, e a soma pode passar do limite do `int64`. Go não avisa quando isso acontece; o valor dá a volta e fica negativo.

Por que `"25.000"` é rejeitado, se vale o mesmo que `"25.00"`?

A escala é fixa em duas casas e o desafio manda rejeitar escala excedente. Aceitar só quando os dígitos extras são zero seria uma exceção difícil de explicar no contrato.

Como o código garante que ninguém altera um `Money` depois de criado?

Os campos não são exportados e os métodos têm receiver por valor, então sempre devolvem um `Money` novo.

O que acontece com `var m Money` seguido de `m.Add(outro)`?

Devolve `ErrInvalidMoney`. O valor zero da struct tem moeda vazia, e isso é tratado como "não inicializado".

Por que `amount` é string no JSON, e não número?

Um número JSON seria lido como ponto flutuante por quase todo parser, inclusive no cliente. Como string, o valor chega exatamente como foi escrito.

Por que `ParseMoney` aceita `"-5.00"`, se o desafio proíbe valor negativo nas entradas?

`Money` também representa diferenças, que podem ser negativas. Quem proíbe negativo em uma aposta ou em um saldo é a entidade que recebe o valor, na spec 03.

SPEC 03

Domínio: Wallet, Ledger e WagerTransaction

As regras financeiras do sistema, em Go puro. Nada aqui conhece banco, HTTP ou fila: são structs, métodos e testes que rodam em milissegundos.

1. O que a spec pedia

- Entidades com estado encapsulado, construtores com validação e métodos explícitos de transição.
- Criação e reidratação separadas. Reidratar não pode movimentar saldo nem transicionar estado.
- Erros classificáveis com `errors.Is` e `errors.As`; sem `panic` para regra de negócio.
- Carteira que nunca fica negativa e cuja versão só sobe quando o saldo muda.
- Lançamento de ledger imutável, que valida `saldoDepois = saldoAntes ± valor`.
- Transação com máquina de estados e as regras de `BET`, `WIN`, `LOSS`, `REFUND` e `ROLLBACK`.
- Um `failureCode` estável para cada rejeição.

2. O que foi construído

ARQUIVO	CONTEÚDO
`errors.go`	Erros sentinela, `ValidationError` e a lista de `FailureCode`
`ledger.go`	`LedgerEntry`: um lançamento de débito ou crédito
`wallet.go`	`Wallet`: saldo, versão, `Debit` e `Credit`
`wager_transaction.go`	`WagerTransaction`: dados da operação e a máquina de estados
`wager_rules.go`	`ApplyWager`: as regras dos cinco tipos

Todos ficam em `internal/domain/`, cada um com o seu `_test.go`.

Como as três peças se relacionam

PEÇA	RESPONDE A	MUDA DEPOIS DE CRIADA?
`WagerTransaction`	O que o provedor pediu e qual foi o resultado?	Só o estado, até ficar terminal
`Wallet`	Quanto o jogador tem agora?	Sim, por `Debit` e `Credit`
`LedgerEntry`	Como o saldo chegou até aqui?	Nunca

3. Wallet

```
type Wallet struct {
    id        string
    playerID  string
    balance   Money
    version   int64
    createdAt time.Time
    updatedAt time.Time
}
```

Todos os campos são privados. O saldo só muda por `Debit` e `Credit`, e os dois devolvem o lançamento de ledger correspondente. Assim é impossível alterar o saldo sem gerar o registro.

```
func (w *Wallet) Debit(entryID, transactionID string, amount Money, now time.Time) (LedgerEntry,
error)
```

Validar tudo antes de alterar

O método calcula o novo saldo, checa as regras, monta o lançamento e só então grava. Se qualquer passo falhar, a carteira fica intacta:

```
if newBalance.IsNegative() {
    return LedgerEntry{}, fmt.Errorf("%w: balance %s, debit %s", ErrInsufficientFunds,
w.balance, amount)
}

entry, err := NewLedgerEntry(entryID, w.id, transactionID, direction, amount, w.balance,
newBalance, now)
if err != nil {
    return LedgerEntry{}, err
}

w.balance = newBalance
w.version++
w.updatedAt = now
return entry, nil
```

Há um teste que executa oito movimentações inválidas e confere, em cada uma, que saldo, versão e data não mudaram.

Criação e reidratação

FUNÇÃO	QUANDO	O QUE FAZ
`OpenWallet`	Carteira nova	Versão 1, saldo inicial, as duas datas iguais
`RehydrateWallet`	Linha lida do banco	Restaura exatamente o que recebeu; só valida

É a diferença entre um construtor público e o que um ORM faria por reflexão em C#. Aqui os dois caminhos são explícitos.

4. LedgerEntry

Um lançamento guarda carteira, transação, direção (`DEBIT` ou `CREDIT`), valor, saldo antes e saldo depois. O construtor refaz a conta e recusa qualquer combinação que não feche:

```
switch direction {
case LedgerDebit:
    expected, err = balanceBefore.Sub(amount)
case LedgerCredit:
    expected, err = balanceBefore.Add(amount)
default:
    return LedgerEntry{}, invalid("direction", "must be DEBIT or CREDIT")
}
...
if !expected.Equal(balanceAfter) {
    return LedgerEntry{}, invalid("balanceAfter", "does not match balanceBefore and amount")
}
```

Não existe método que altere um `LedgerEntry` . Para corrigir um erro financeiro, cria-se um novo lançamento no sentido contrário. É o que "append-only" significa.

5. WagerTransaction e a máquina de estados

ESTADO	SIGNIFICADO	PODE IR PARA
`PENDING`	Aceita, ainda não concluída	`PENDING_REFERENCE` , `PROCESSED` , `REJECTED` , `FAILED`
`PENDING_REFERENCE`	Esperando a operação referenciada chegar	`PROCESSED` , `REJECTED` , `FAILED`
`PROCESSED`	Concluída com sucesso	Nenhum (terminal)
`REJECTED`	Recusada por regra de negócio	Nenhum (terminal)
`FAILED`	Falha permanente de infraestrutura	Nenhum (terminal)

No código, a tabela acima é um mapa, e toda mudança de estado passa por uma única função:

```
var allowedTransitions = map[TransactionStatus][]TransactionStatus{
    StatusPending:          {StatusPendingReference, StatusProcessed, StatusRejected,
StatusFailed},
    StatusPendingReference: {StatusProcessed, StatusRejected, StatusFailed},
}

func (t *WagerTransaction) transition(to TransactionStatus, now time.Time) error {
    ...
    if !slices.Contains(allowedTransitions[t.status], to) {
        return fmt.Errorf("%w: %s -> %s", ErrInvalidTransition, t.status, to)
    }
    t.status = to
    t.updatedAt = now
    return nil
}
```

Os estados terminais não aparecem como chave do mapa. Buscar uma chave ausente devolve uma lista vazia, então toda transição a partir deles é recusada, sem precisar de um `if` para cada caso.

Os métodos públicos (`MarkProcessed`, `MarkRejected`, `MarkFailed`, `MarkPendingReference`) chamam `transition` e gravam o que acompanha cada resultado: o saldo observado ou o código de falha.

Validação por tipo, na criação

TIPO	VALOR	REFERÊNCIA
`BET`	Maior que zero	Não aceita
`WIN`	Maior que zero	Opcional
`LOSS`	Exatamente `0.00`	Não aceita
`REFUND`	Maior que zero	Obrigatória
`ROLLBACK`	Maior que zero	Obrigatória
`OPENING`	Recusado em `NewWagerTransaction`; só existe por `NewOpeningTransaction`	

A abertura de carteira tem construtor próprio. Ela já nasce `PROCESSED` e não carrega provedor, ID externo, chave, hash, rodada nem jogo.

6. ApplyWager: as regras dos cinco tipos

```
func ApplyWager(tx *WagerTransaction, wallet *Wallet, ref Reference, entryID string, now
time.Time) (*LedgerEntry, error)
```

Recebe a transação, a carteira e o que foi encontrado sobre a referência. Decide o resultado e o grava nos dois objetos.

1. A carteira é do mesmo jogador? Se não: `PLAYER_MISMATCH`.
2. A moeda é a da carteira? Se não: `CURRENCY_MISMATCH`.
3. Tem referência? Confere contexto, tipo, estado, valor e se já foi revertida.

4. Referência ainda não chegou ou está pendente: a transação fica `PENDING_REFERENCE`.
5. Define o movimento: débito, crédito ou nenhum.
6. Movimenta a carteira. Falta de saldo vira rejeição.
7. Marca `PROCESSED` com o saldo resultante.

TIPO	MOVIMENTO	REFERÊNCIA ACEITA
`BET`	Débito	—
`WIN`	Crédito	`BET`
`LOSS`	Nenhum; a carteira não é tocada	—
`REFUND`	Crédito	`BET`
`ROLLBACK`	De `BET`: crédito. De `WIN` ou `REFUND`: débito	`BET`, `WIN`, `REFUND`

> **Rejeição não é erro.** Aposta sem saldo é um resultado de negócio esperado. `ApplyWager` retorna sem erro, e a transação fica `REJECTED` com um `failureCode`. Isso importa porque a rejeição precisa ser gravada no banco e devolvida igual em um reenvio. O retorno de erro fica para uso incorreto do código, como aplicar uma transação já terminal.

Dois tipos de "não deu certo"

	ENTRADA INVÁLIDA	REJEIÇÃO DE NEGÓCIO
Exemplo	`BET` com valor zero	`BET` sem saldo
Onde é detectada	No construtor	Em `ApplyWager`
Como aparece no código	Erro do tipo `*ValidationError`	Transação `REJECTED` + `failureCode`
É gravada?	Não	Sim
O provedor pode corrigir e reenviar?	Sim	Não; é definitivo
HTTP (spec 07)	400	422

Códigos de falha

CÓDIGO	QUANDO
`INSUFFICIENT_FUNDS`	`BET` sem saldo
`REVERSAL_INSUFFICIENT_FUNDS`	`ROLLBACK` que debitaria mais que o saldo
`BALANCE_LIMIT_EXCEEDED`	Crédito que passaria do limite do `int64`
`PLAYER_MISMATCH`	A carteira é de outro jogador
`CURRENCY_MISMATCH`	Moeda diferente da carteira
`REFERENCE_NOT_FOUND`	A referência não chegou até expirar (spec 08)
`REFERENCE_NOT_PROCESSED`	A referência terminou rejeitada ou com falha
`REFERENCE_MISMATCH`	Diverge em provedor, jogador, carteira, moeda ou rodada
`REFERENCE_AMOUNT_MISMATCH`	Valor da reversão diferente do original
`REFERENCE_KIND_NOT_ALLOWED`	Por exemplo, `REFUND` de um `WIN`
`ALREADY_REVERSED`	A referência já tem uma reversão processada

7. Conceitos de Go

Tipo de erro próprio e `errors.As`

O erro sentinela diz *qual* erro aconteceu. Quando o erro precisa carregar dados, usa-se uma struct que implementa o método `Error()`:

```
type ValidationError struct {
    Field  string
    Reason string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("invalid %s: %s", e.Field, e.Reason)
}
```

Quem recebe usa `errors.As` para recuperar a struct e ler os campos. É o `catch (ValidationException ex)` seguido de `ex.Field`:

```
var validation *ValidationError
if errors.As(err, &validation) {
    // validation.Field, validation.Reason
}
```

	`errors.Is`	`errors.As`
Pergunta	É este erro específico?	É um erro deste tipo?
Compara com	Uma variável (`ErrInsufficientFunds`)	Um tipo (`*ValidationError`)
Dá acesso a dados	Não	Sim

Não há `implements` : `ValidationError` vale como `error` só por ter o método `Error() string` . É a interface implícita do Go.

Enum como tipo nomeado

Go não tem `enum` . O idioma é criar um tipo a partir de `string` e declarar as constantes dele:

```
type TransactionKind string

const (
    KindOpening  TransactionKind = "OPENING"
    KindBet      TransactionKind = "BET"
    ...
)

func (k TransactionKind) IsReversal() bool {
    return k == KindRefund || k == KindRollback
}
```

O compilador impede passar uma `string` comum onde se espera `TransactionKind` , e o tipo pode ter métodos. Mas ele não impede um valor desconhecido como `TransactionKind("DEPOSIT")` , por isso os construtores validam.

Ponteiro ou valor

TIPO	RECEIVER	MOTIVO
`Money` , `LedgerEntry`	Valor: `(m Money)`	Imutáveis; copiar é seguro e barato
`Wallet` , `WagerTransaction`	Ponteiro: `(w *Wallet)`	Têm identidade e mudam de estado; todos precisam ver o mesmo objeto

É a distinção entre `struct` e `class` do C#, só que escolhida em cada método.

Struct de parâmetros

Go não tem argumentos nomeados nem opcionais. Para uma função com muitos parâmetros do mesmo tipo, agrupa-se tudo em uma struct, o que evita trocar a ordem por engano:

```
tx, err := domain.NewWagerTransaction(domain.NewWagerTransactionParams{
    ID:         "...",
    ProviderID: "provider-a",
    Kind:       domain.KindBet,
    Money:      money,
    Now:        now,
})
```

Outros pontos

- `switch` **sem expressão** funciona como uma cadeia de `if / else if` e não precisa de `break`.
- **Closure:** em `ApplyWager`, `reject` é uma função declarada dentro de outra, que enxerga `tx`, `wallet` e `now`.
- **Retorno `(valor, bool)`:** `ResultBalance()` devolve o saldo e se ele existe. É o padrão do Go para "pode não ter valor", no lugar de `null`.
- `time.Time{}` é o valor zero de data; `IsZero()` detecta data não informada.

8. Decisões

DECISÃO	MOTIVO
IDs e o instante entram como parâmetro	O domínio não gera UUID nem lê o relógio: testes determinísticos e nenhuma dependência externa
`Debit` e `Credit` devolvem o lançamento	Saldo e ledger nascem juntos; não há como mudar um sem o outro
Rejeição de negócio é resultado, não erro	Precisa ser persistida e reproduzida em reenvios
Uma transação aceita uma única reversão processada, de qualquer tipo	Impede devolver duas vezes o mesmo débito (`REFUND` + `ROLLBACK` da mesma aposta)
`ROLLBACK` de `ROLLBACK` é recusado	O desafio lista só `BET`, `WIN` e `REFUND` como alvos
`WIN` com referência espera a aposta chegar	Mesma regra das reversões; um caminho só para referência ausente
Código próprio para valor divergente (`REFERENCE_AMOUNT_MISMATCH`)	É o erro mais provável de integração; fica fácil de diagnosticar
`LedgerEntry` tem um construtor só	Não há comportamento a reaplicar na reidratação, apenas a equação a conferir

> **O que esta spec ainda não garante.** As regras estão corretas para um processo só, uma operação por vez. Duas apostas *simultâneas*, em instâncias diferentes, carregariam cada uma a sua cópia da carteira com saldo 100. Quem resolve isso é o lock no banco (spec 05) e as constraints (spec 04). O domínio decide; o banco garante.

9. Como foi verificado

VERIFICAÇÃO	RESULTADO
`go test ./internal/domain/ -cover`	243 casos passam; 99,7% das instruções
`go test -race ./...` (em container)	Passou
`gofmt -l .` e `go vet ./...`	Sem saída
Imports do pacote `domain`	Só biblioteca padrão

O que os testes cobrem

- **Carteira:** abertura, reidratação, débito, crédito, e oito movimentações inválidas que não podem alterar o estado.
- **Ledger:** a equação do saldo nas duas direções e onze entradas inválidas.
- **Máquina de estados:** as 20 combinações de 5 estados de origem por 4 destinos.
- **Política por tipo:** 15 combinações de tipo e valor, 10 de tipo e referência.
- **Regras:** cada tipo no caminho feliz, as duas apostas de 80 sobre 100, a reversão que chega antes da referência e 10 rejeições por referência.

10. Perguntas de revisão

Por que `Debit` devolve um `LedgerEntry` em vez de só alterar o saldo?

Para que seja impossível mudar o saldo sem produzir o lançamento. O desafio exige que cada mudança financeira tenha o registro correspondente.

Uma aposta sem saldo faz `ApplyWager` devolver erro?

Não. A função retorna sem erro e a transação fica `REJECTED` com `INSUFFICIENT_FUNDS`. É um resultado de negócio que será gravado.

Qual a diferença entre `errors.Is` e `errors.As`?

`Is` compara com um erro específico (uma variável). `As` procura um erro de um tipo e devolve a struct, dando acesso aos campos.

Por que existem `OpenWallet` e `RehydrateWallet`?

Criar define o estado inicial (versão 1). Reidratar restaura o que estava gravado, sem aplicar nenhuma regra de criação.

Como o código impede uma transição a partir de `PROCESSED`?

`PROCESSED` não é chave no mapa de transições. A busca devolve lista vazia e `transition` retorna `ErrInvalidTransition`.

Um `ROLLBACK` debita ou credita?

Depende do original: faz o movimento contrário. De uma `BET` (débito), credita. De um `WIN` ou `REFUND` (créditos), debita.

O domínio sozinho impede saldo negativo com duas instâncias rodando?

Não. Cada instância teria a sua cópia da carteira em memória. A garantia entre processos vem do lock e das constraints do banco.

Por que `LOSS` não altera a versão da carteira?

A versão só sobe quando o saldo muda. `LOSS` não movimenta nada, então `ApplyWager` nem chama a carteira.

Ordem de leitura do código

1. `errors.go`
2. `ledger.go`
3. `wallet.go` e `wallet_test.go`
4. `wager_transaction.go`
5. `wager_rules.go` e `wager_rules_test.go`

SPEC 04

Schema e migrations

As mesmas regras do domínio, agora escritas no banco. Se a aplicação tiver um bug, ou se duas instâncias disputarem a mesma carteira, o PostgreSQL recusa o que quebraria uma invariante.

1. O que a spec pedia

- Migrations versionadas, com aplicação e reversão.
- Saldo nunca negativo, garantido pelo banco.
- Unicidade da carteira por jogador e moeda, da operação externa e da chave de idempotência.
- Ledger com um lançamento por transação e carteira, protegido contra edição e exclusão.
- Schema que distingue operação interna (`OPENING`) de externa e impede crédito inicial duplicado.
- Tabelas de inbox e outbox.

2. O que foi construído

ARQUIVO	CONTEÚDO
`migrations/000001_create_wallets`	Tabela `wallets`
`migrations/000002_create_wager_transactions`	Tabela `wager_transactions` e seus índices únicos parciais
`migrations/000003_create_wallet_ledger_entries`	Tabela do ledger e os triggers que a tornam append-only
`migrations/000004_create_inbox_and_outbox`	Tabelas `inbox_messages` e `outbox_events`
`migrations/migrations_test.go`	Teste que aplica as migrations e tenta violar cada invariante
`docker-compose.yml`	Serviço `migrate` , que roda antes da aplicação

Cada migration tem dois arquivos: `.up.sql` (aplica) e `.down.sql` (desfaz). O número no início do nome define a ordem.

3. Como as migrations funcionam

A ferramenta é o `golang-migrate` . Ela cria no banco a tabela `schema_migrations` , com uma linha: a versão atual. `up` aplica os arquivos com número maior que essa versão; `down` roda os `.down.sql` na ordem inversa.

É o mesmo papel das migrations do Entity Framework ou do TypeORM, com uma diferença: aqui o SQL é escrito à mão, não gerado a partir de classes.

1. `docker compose up` sobe o Postgres e espera o healthcheck.
2. O serviço `migrate` aplica o que falta e termina.

3. O serviço `app` só inicia depois que o `migrate` termina com sucesso (`service_completed_successfully`).

COMANDO	EFEITO
`docker compose run --rm migrate`	Aplica tudo o que falta
`... migrate -path=/migrations -database=<url> down 1`	Reverte a última migration
`... down -all`	Reverte todas
`... version`	Mostra a versão atual

4. As tabelas

TABELA	UMA LINHA É	PODE MUDAR?
`wallets`	Uma carteira e seu saldo atual	Sim: saldo, versão e `updated_at`
`wager_transactions`	Uma operação, interna ou externa	Sim: estado, código de falha, saldo resultante
`wallet_ledger_entries`	Um débito ou crédito	Nunca
`inbox_messages`	Uma mensagem SQS já recebida por um consumidor	Só a conclusão
`outbox_events`	Um evento a publicar	Só os campos de envio

Dinheiro é sempre um par de colunas: `BIGINT` com os centavos e `CHAR(3)` com a moeda. É a mesma representação do `Money` da spec 02, então gravar e ler não converte nada.

5. Os quatro mecanismos usados

CHECK: regra sobre uma linha

```
CONSTRAINT wallets_balance_non_negative CHECK (balance_minor >= 0)
```

O banco avalia a expressão em todo `INSERT` e `UPDATE`. Se der falso, o comando falha com SQLSTATE `23514`. Esta é a rede de segurança contra saldo negativo: mesmo que o lock da aplicação falhasse, o `UPDATE` não passaria.

O `CHECK` enxerga só a própria linha. Ele serve para regras como "`LOSS` tem valor zero" ou "transação rejeitada tem código de falha", mas não para "só existe uma abertura por carteira".

```
CONSTRAINT wallet_ledger_entries_balance_equation CHECK (
    (direction = 'CREDIT' AND balance_after_minor = balance_before_minor + amount_minor)
    OR (direction = 'DEBIT' AND balance_after_minor = balance_before_minor - amount_minor)
)
```

É a mesma equação que `NewLedgerEntry` valida no domínio. A regra existe nos dois lugares de propósito.

UNIQUE e índice único parcial: regra entre linhas

```
CONSTRAINT wager_transactions_external_key UNIQUE (provider_id, external_transaction_id)
```

Duas linhas com o mesmo par falham com SQLSTATE `23505`. É isso que impede a mesma operação de ser aplicada duas vezes quando duas instâncias a recebem ao mesmo tempo: uma insere, a outra recebe o erro.

Um índice único **parcial** aplica a unicidade só às linhas que passam em um filtro:

```
CREATE UNIQUE INDEX wager_transactions_one_opening_per_wallet
    ON wager_transactions (wallet_id)
    WHERE kind = 'OPENING';

CREATE UNIQUE INDEX wager_transactions_one_processed_reversal
    ON wager_transactions (reference_transaction_id)
    WHERE status = 'PROCESSED' AND kind IN ('REFUND', 'ROLLBACK');
```

O primeiro permite muitas transações por carteira, mas só uma `OPENING`. O segundo permite várias reversões rejeitadas da mesma aposta, mas só uma processada. É a regra "uma reversão por transação" da spec 03, agora à prova de concorrência.

> **NULL em UNIQUE.** O Postgres trata cada `NULL` como diferente dos outros. Por isso as transações `OPENING`, que têm `provider_id` nulo, não colidem entre si na constraint `(provider_id, external_transaction_id)`.

Chave estrangeira composta: coerência entre tabelas

```
FOREIGN KEY (wallet_id, currency)       REFERENCES wallets (id, currency),
FOREIGN KEY (transaction_id, wallet_id) REFERENCES wager_transactions (id, wallet_id)
```

Uma FK comum garante que a carteira existe. A composta garante mais: que o lançamento está na **moeda da carteira** e pertence à **mesma carteira da transação**. Para isso a tabela de destino precisa de um `UNIQUE` nas mesmas colunas, e é por isso que `wallets` tem `UNIQUE (id, currency)`, que parece redundante.

Trigger: o ledger append-only

```
CREATE FUNCTION reject_wallet_ledger_entry_change() RETURNS trigger
    LANGUAGE plpgsql AS
$$
BEGIN
    RAISE EXCEPTION 'wallet_ledger_entries is append-only: % is not allowed', TG_OP
        USING ERRCODE = 'restrict_violation';
END;
$$;

CREATE TRIGGER wallet_ledger_entries_append_only
    BEFORE UPDATE OR DELETE ON wallet_ledger_entries
    FOR EACH ROW EXECUTE FUNCTION reject_wallet_ledger_entry_change();
```

Um trigger é uma função que o banco executa quando algo acontece na tabela. Esta roda antes de cada `UPDATE` ou `DELETE` e sempre lança erro; `TG_OP` é o nome da operação tentada. Só `INSERT` passa.

Há um segundo trigger para `TRUNCATE`, porque o trigger por linha não dispara nesse comando.

6. Interno e externo na mesma tabela

`OPENING` não tem provedor, ID externo, chave, hash, rodada nem jogo. As outras operações têm todos. As colunas são anuláveis e um `CHECK` exige um dos dois formatos, sem meio-termo:

```
CONSTRAINT wager_transactions_origin CHECK (
    (kind = 'OPENING' AND status = 'PROCESSED'
        AND num_nonnulls(provider_id, external_transaction_id, ...) = 0)
    OR
    (kind <> 'OPENING'
        AND num_nulls(provider_id, external_transaction_id, ...) = 0)
)
```

`num_nulls` e `num_nonnulls` contam quantos argumentos são nulos ou não nulos. Evitam uma fila de `IS NULL AND ... IS NULL`.

7. Domínio e banco: quem garante o quê

REGRA	NO DOMÍNIO	NO BANCO
Saldo não negativo	`Wallet.Debit`	`CHECK`
Equação do lançamento	`NewLedgerEntry`	`CHECK`
Valor e referência por tipo	`NewWagerTransaction`	`CHECK`
Uma reversão processada	`Reference.Reversed`	Índice único parcial
Operação externa única	—	`UNIQUE`
Uma carteira por jogador e moeda	—	`UNIQUE`
Ledger imutável	Struct sem setters	Trigger

> **Por que repetir no banco.** Com três instâncias, cada uma tem a sua cópia dos objetos em memória. A validação do domínio só enxerga essa cópia. O banco é o único lugar que todas as instâncias compartilham, então é o único que pode dizer "isso já existe".

O domínio continua necessário: ele dá a mensagem de erro certa e o `failureCode` antes de chegar ao banco. A constraint cobre o que escapar.

8. Decisões

DECISÃO	MOTIVO
`kind`, `status` e `direction` em `TEXT` + `CHECK`, não `ENUM`	Mudar a lista de um `CHECK` é reversível; remover um valor de `ENUM` do Postgres não é
IDs internos em `UUID`, identificadores de fora em `TEXT`	O formato de `providerId` ou `externalTransactionId` é do provedor
Coluna `sequence` (identity) no ledger	Ordem estável para a paginação. `created_at` pode empatar; a sequência não
Sem `DEFAULT now()`	O instante vem da aplicação, o mesmo que o domínio usou
Constraints com nome explícito	O nome vem no erro do Postgres, o que permite à spec 05 traduzir cada violação em um erro de aplicação
`locked_until` na outbox	Um publisher reserva o evento por um prazo; se morrer, outro assume (spec 10)
Quatro migrations em vez de uma	Cada arquivo trata de um assunto e pode ser revertido sozinho

9. O teste

Constraint não se testa com mock: o teste precisa de um PostgreSQL real. Ele lê a URL de `TEST_DATABASE_URL` e é pulado quando ela não existe.

```
func connect(t *testing.T) *pgx.Conn {
    url := os.Getenv("TEST_DATABASE_URL")
    if url == "" {
        t.Skip("TEST_DATABASE_URL is not set")
    }
    // cria um schema descartável e aponta o search_path para ele
    t.Cleanup(func() { /* DROP SCHEMA ... CASCADE */ })
}
```

- `t.Skip` marca o teste como pulado, não como falho.
- `t.Cleanup` registra uma função para rodar ao fim do teste. É o `afterEach` do Jest ou o `Dispose` do xUnit.
- Cada teste cria um **schema** próprio (um namespace de tabelas). Assim ele não interfere nas tabelas reais nem em outro teste.

Cada violação é conferida pelo código e pelo nome da constraint, não só por "deu erro":

```
var pgErr *pgconn.PgError
if !errors.As(err, &pgErr) { ... }
// pgErr.Code           == "23514"
// pgErr.ConstraintName == "wallets_balance_non_negative"
```

É o `errors.As` da spec 03 em uso real: extrai do erro a struct do driver, com o SQLSTATE dentro.

SQLSTATE	SIGNIFICADO
`23505`	Violação de `UNIQUE`
`23514`	Violação de `CHECK`
`23503`	Violação de chave estrangeira
`23001`	`restrict_violation`: o código escolhido para o trigger do ledger

10. Como foi verificado

VERIFICAÇÃO	RESULTADO
`migrate up` → `down -all` → `up`	Depois do `down` sobra só `schema_migrations`; o segundo `up` aplica as 4
`go test ./migrations/` com banco real	22 violações recusadas e 5 casos válidos aceitos
`go test -race ./...` (em container)	Passou
`docker compose up --build`	`migrate` termina com código 0, depois `app` sobe e `/health/ready` responde 200

```
docker compose up -d postgres
docker compose run --rm migrate
TEST_DATABASE_URL='postgres://wagering:wagering@localhost:5432/wagering?sslmode=disable' go test
./migrations/
```

11. Perguntas de revisão

Se o domínio já impede saldo negativo, para que serve o `CHECK`?

O domínio valida a cópia em memória de uma instância. O `CHECK` vale para todas as instâncias e para qualquer bug da aplicação.

Qual a diferença entre `UNIQUE` e índice único parcial?

O parcial tem um `WHERE`: a unicidade vale só para as linhas que passam no filtro. Ex.: uma única `OPENING` por carteira.

Por que duas `OPENING` não colidem em `UNIQUE (provider_id, external_transaction_id)`?

Os dois campos são `NULL` nelas, e o Postgres não considera dois `NULL` iguais.

Como o ledger fica imutável?

Um trigger `BEFORE UPDATE OR DELETE` lança erro em toda tentativa, e outro faz o mesmo para `TRUNCATE`.

O que o `CHECK` não consegue fazer?

Comparar com outras linhas ou tabelas. Para isso existem `UNIQUE`, chave estrangeira e trigger.

Para que serve `UNIQUE (id, currency)` em `wallets`, se `id` já é único?

Para ser o destino da FK composta do ledger, que garante lançamento na moeda da carteira.

Como o `app` sabe que o schema está pronto?

No Compose ele depende do `migrate` com `service_completed_successfully`.

Por que o teste de schema é pulado no `go test ./...` comum?

Sem `TEST_DATABASE_URL` ele chama `t.Skip`. Ele precisa de um Postgres real.

Ordem de leitura

1. `000001_create_wallets.up.sql`
2. `000002_create_wager_transactions.up.sql`
3. `000003_create_wallet_ledger_entries.up.sql`
4. `000004_create_inbox_and_outbox.up.sql`
5. `migrations_test.go`: a lista `rejected` é um resumo executável das invariantes

SPEC 05

Persistência e transação SQL

A ponte entre o domínio e o banco: repositórios com SQL escrito à mão, uma transação compartilhada por todos eles e o lock por carteira que impede duas instâncias de gastarem o mesmo saldo.

1. O que a spec pedia

- `pgx` com SQL explícito; transações, locks e constraints visíveis no código.
- Um único mecanismo que delimita a transação SQL, compartilhada por todos os repositórios de um caso de uso.
- Lock por carteira. Lock global é proibido; carteiras diferentes avançam em paralelo.
- Nenhuma atualização confirmada pode ser perdida.
- Todo I/O recebe `context.Context` e respeita cancelamento.
- Falha transitória distinguível de falha permanente.

2. O que foi construído

ARQUIVO	CONTEÚDO
`internal/application/ports.go`	As interfaces dos repositórios e os erros que os casos de uso enxergam
`internal/infra/postgres/tx.go`	`TxManager` : abre, confirma ou desfaz a transação
`internal/infra/postgres/errors.go`	Traduz erro do PostgreSQL para erro de aplicação
`wallet_repository.go`	Carteira: inserir, ler, ler com lock, atualizar saldo
`wager_transaction_repository.go`	Transação: inserir, atualizar, buscar por ID, por ID externo e por chave
`ledger_repository.go` , `outbox_repository.go`	Só inserção
`internal/testdb/testdb.go`	Banco de teste: um schema migrado por teste

3. Ports: a interface fica com quem usa

```
// internal/application/ports.go
type WalletRepository interface {
    Insert(ctx context.Context, wallet *domain.Wallet) error
    Get(ctx context.Context, id string) (*domain.Wallet, error)
    GetForUpdate(ctx context.Context, id string) (*domain.Wallet, error)
    UpdateBalance(ctx context.Context, wallet *domain.Wallet) error
}
```

A interface está no pacote `application`, que é quem **consome** o repositório. A implementação está em `infra/postgres`. Assim `application` não importa nada de banco, e a regra de dependência do desafio se mantém.

Em Go a implementação não declara que implementa. `*postgres.WalletRepository` satisfaz a interface só por ter os mesmos métodos. Em C# seria `class WalletRepository : IWalletRepository`; aqui não existe essa linha.

Quem liga as duas pontas é o Fx:

```
fx.Annotate(NewWalletRepository, fx.As(new(application.WalletRepository)))
```

`fx.As` registra o resultado do construtor **como a interface**. É o `services.AddScoped<IWalletRepository, WalletRepository>()` do C#. Um caso de uso vai pedir `application.WalletRepository` e receber a implementação do Postgres sem conhecê-la.

4. A transação viaja no context

```
func (m *TxManager) WithinTx(ctx context.Context, fn func(ctx context.Context) error) error {
    if _, ok := ctx.Value(txContextKey{}).(pgx.Tx); ok {
        return fn(ctx)                      // já existe transação: entra nela
    }

    tx, err := m.pool.Begin(ctx)
    if err != nil {
        return translate(err)
    }
    defer func() { _ = tx.Rollback(context.WithoutCancel(ctx)) }()

    if err := fn(context.WithValue(ctx, txContextKey{}, tx)); err != nil {
        return err                          // o defer faz o rollback
    }
    return translate(tx.Commit(ctx))
}
```

1. `Begin` pega uma conexão do pool e abre a transação.
2. `context.WithValue` cria um novo `ctx` que carrega a transação.
3. A função do caso de uso recebe esse `ctx` e o repassa a cada repositório.
4. Cada repositório procura a transação no `ctx`. Achou: usa ela. Não achou: usa o pool.
5. Retornou `nil`: commit. Retornou erro: rollback.

```
func querierFrom(ctx context.Context, pool *pgxpool.Pool) querier {
    if tx, ok := ctx.Value(txContextKey{}).(pgx.Tx); ok {
        return tx
    }
    return pool
}
```

O caso de uso não vê `pgx.Tx` em lugar nenhum. Ele só escreve:

```
err := txManager.WithinTx(ctx, func(ctx context.Context) error {
    wallet, err := wallets.GetForUpdate(ctx, walletID)
    // ... regra do domínio ...
    if err := wagers.Insert(ctx, tx); err != nil { return err }
    if err := wallets.UpdateBalance(ctx, wallet); err != nil { return err }
    return ledger.Insert(ctx, *entry)
})
```

É o papel do `TransactionScope` do C# ou do `@Transactional`, feito com um parâmetro explícito em vez de estado escondido na thread.

Detalhes do WithinTx

TRECHO	POR QUÊ
`txContextKey struct{}`	A chave do `context` é um tipo privado do pacote. Nenhum outro pacote consegue ler ou sobrescrever o valor
`ctx.Value(...).(pgx.Tx)`	Type assertion: `Value` devolve `any`; a forma com `, ok` não dá panic se o valor não existir
`defer tx.Rollback(...)`	Roda em qualquer saída, inclusive panic. Depois de um commit não faz nada
`context.WithoutCancel(ctx)`	Se o `ctx` foi cancelado, o rollback com ele também falharia. Este mantém os valores e remove o cancelamento
Interface `querier`	Pool e transação têm os mesmos métodos `Exec`, `Query` e `QueryRow`. O repositório depende só deles

5. O lock por carteira

```
SELECT id::text, player_id, currency, balance_minor, version, created_at, updated_at
  FROM wallets
 WHERE id = $1
   FOR UPDATE
```

`FOR UPDATE` trava **a linha** até o fim da transação. Outra transação que pedir a mesma linha com `FOR UPDATE` fica parada esperando. Quando a primeira termina, a segunda lê o saldo já atualizado.

INSTANTE	INSTÂNCIA A (APOSTA DE 80)	INSTÂNCIA B (APOSTA DE 80)
1	`GetForUpdate`: lê saldo 100	
2		`GetForUpdate`: **espera**
3	Debita, grava saldo 20, commit	
4		Lê saldo **20**
5		Domínio rejeita: `INSUFFICIENT_FUNDS`

Sem o lock, as duas leriam 100 e as duas gravariam 20: uma aposta sumiria do saldo. Esse é o *lost update*.

O lock é da linha, não da tabela. Uma aposta em outra carteira não espera ninguém.

Duas proteções extras

```
UPDATE wallets
   SET balance_minor = $2, version = $3, updated_at = $4
 WHERE id = $1 AND version = $3 - 1
```

- **Guarda de versão.** O `UPDATE` só acerta a linha se a versão no banco for a anterior à nova. Se alguém gravou sem o lock, nenhuma linha é afetada e o repositório devolve `ErrConcurrentUpdate`.
- `GetForUpdate` **fora de transação retorna erro.** Sem transação o lock seria solto no mesmo instante.

Com o `CHECK (balance_minor >= 0)` da spec 04, são três camadas contra saldo errado: lock, guarda de versão e constraint.

6. SQL e mapeamento

Sem ORM. Cada repositório escreve o SQL e converte as colunas na mão.

```
err := row.Scan(&id, &playerID, &currency, &balanceMinor, &version, &createdAt, &updatedAt)
if errors.Is(err, pgx.ErrNoRows) {
    return nil, application.ErrWalletNotFound
}
balance, err := domain.MoneyFromMinorUnits(balanceMinor, currency)
return domain.RehydrateWallet(id, playerID, balance, version, createdAt.UTC(), updatedAt.UTC())
```

- `Scan` recebe ponteiros (`&id`) e preenche as variáveis, na ordem das colunas do `SELECT`.
- A leitura termina em `RehydrateWallet`, a função de reidratação da spec 03: restaura o estado sem aplicar regra de criação.
- Parâmetros são `$1`, `$2` ..., nunca concatenação de string. É o que evita SQL injection.

NO DOMÍNIO	NO BANCO	CONVERSÃO
`Money`	`BIGINT` + `CHAR(3)`	`MinorUnits()` / `MoneyFromMinorUnits`
Campo opcional vazio (`""`)	`NULL`	`NULLIF($8, '')` na escrita, `COALESCE(col, '')` na leitura
Saldo resultante não observado	`NULL`	Ponteiro `*int64` : `nil` vira `NULL`
`string` com UUID	`UUID`	`id::text` na leitura

> **Ponteiro como "pode ser nulo".** `int64` não tem valor nulo: o zero é `0`. Para representar ausência usa-se `*int64`, que pode ser `nil`. É o `long?` do C#.

7. Tradução de erros

O caso de uso não pode depender de códigos do PostgreSQL. `translate` converte cada erro em um erro de `application`:

DO POSTGRESQL	VIRA	TIPO
`23505` em `wallets_player_currency_key`	`ErrWalletAlreadyExists`	Permanente
`23505` em `wager_transactions_external_key`	`ErrDuplicateExternalTransaction`	Permanente
`23505` em `wager_transactions_idempotency_key`	`ErrDuplicateIdempotencyKey`	Permanente
Outras da classe `23`	`ErrInvariantViolation`	Permanente (é bug)
`40001`, `40P01`, classes `08`, `53`, `57`, erro de rede	`ErrUnavailable`	**Transitório**

É aqui que os nomes explícitos das constraints da spec 04 são usados: o mesmo código `23505` vira erros diferentes conforme a constraint violada.

```
return fmt.Errorf("%w: %w", application.ErrDuplicateIdempotencyKey, pgErr)
```

Dois `%w` encadeiam os dois erros. `errors.Is(err, application.ErrDuplicateIdempotencyKey)` é verdadeiro, e o erro original do Postgres continua acessível para o log.

> **Transitório × permanente.** Transitório: repetir a mesma chamada mais tarde pode funcionar (banco reiniciando, deadlock). Vai virar `503` no HTTP e reentrega no SQS. Permanente: repetir dá o mesmo resultado. Essa distinção é exigida pelo desafio e decide o que vai para a DLQ.

Depois de uma violação de constraint, o PostgreSQL marca a transação como abortada: qualquer comando seguinte falha. Por isso o caso de uso retorna o erro em vez de tentar continuar.

8. Testes

O pacote `testdb` cria um schema novo para cada teste, aplica as migrations nele e apaga tudo no fim. Os testes rodam em paralelo com o banco de desenvolvimento sem interferir.

```
config.ConnConfig.RuntimeParams["search_path"] = schema
```

Toda conexão do pool de teste já nasce apontando para o schema descartável.

O teste do lock

```
locked := make(chan struct{})
release := make(chan struct{})
go func() {
    holderDone <- f.tx.WithinTx(ctx, func(ctx context.Context) error {
        f.wallets.GetForUpdate(ctx, id(1))
        close(locked)      // avisa: estou com o lock
        <-release          // segura até mandarem soltar
        return nil
    })
}()
<-locked
```

- `go func() { ... }()` inicia uma goroutine: a função roda em paralelo.
- Um **channel** é um canal de comunicação entre goroutines. `<-locked` bloqueia até alguém enviar ou fechar o canal.
- `close(locked)` libera todos que estão esperando. Canais de `struct{}` servem só como sinal, sem dado.

Com o lock preso, o teste tenta travar a mesma carteira com prazo de 300 ms e espera `context.DeadlineExceeded`. Isso prova duas coisas de uma vez: a segunda transação espera, e o cancelamento do `context` interrompe a query.

O teste de concorrência

```
var wg sync.WaitGroup
for n := range bets {
    wg.Go(func() { /* uma aposta de 1.00 */ })
}
wg.Wait()
```

`sync.WaitGroup` espera um grupo de goroutines terminar: é o `Task.WhenAll`. Vinte apostas de 1.00 disputam uma carteira de 20.00.

9. Como foi verificado

VERIFICAÇÃO	RESULTADO
Erro dentro de `WithinTx`	Carteira, transação, ledger e outbox ficam com 0 linhas; com sucesso, 1 em cada
Mesma carteira travada	Segunda transação espera e é cortada pelo prazo; outra carteira trava sem esperar
20 apostas simultâneas de 1.00 sobre 20.00	Saldo 0.00, versão 21, 20 lançamentos; a 21ª é rejeitada
Gravação sem lock	`ErrConcurrentUpdate`
Ida e volta de `Money`	Valor e moeda idênticos, inclusive no maior saldo representável
`go test -race` (em container, 3 repetições)	Passou
Cobertura do pacote `postgres`	83,6%

O teste de concorrência roda dentro de **um** processo, com várias conexões. A prova com três processos independentes, exigida pelo desafio, fica na spec 15.

10. Uma correção no schema

Ao escrever o repositório apareceu um furo da spec 04: o saldo resultante tinha só a coluna de valor. Em uma rejeição por `CURRENCY_MISMATCH`, o saldo devolvido ao provedor está na moeda da carteira, que é diferente da moeda da transação. Sem a moeda gravada, um replay devolveria o saldo com a moeda errada.

A coluna `result_balance_currency` foi acrescentada, com um `CHECK` que exige as duas colunas juntas. O teste de ida e volta cobre exatamente esse caso.

11. Perguntas de revisão

Como um repositório sabe que está dentro de uma transação?

Ele procura a transação no `context`. O `TxManager` a colocou lá com `context.WithValue`.

Por que a interface `WalletRepository` fica em `application` e não em `postgres`?

Porque quem usa define o contrato. Assim `application` não importa nada de banco.

O que `FOR UPDATE` trava, e até quando?

As linhas retornadas pelo `SELECT`, até o commit ou rollback da transação.

O que é lost update e como é evitado aqui?

Duas transações leem o mesmo saldo e uma sobrescreve a outra. O lock faz a segunda ler só depois do commit da primeira.

Para que serve a guarda de versão se já existe o lock?

Para transformar um uso errado (gravar sem lock) em erro, em vez de corromper o saldo em silêncio.

Por que o rollback usa `context.WithoutCancel`?

Se o `ctx` original foi cancelado, o rollback feito com ele também seria cancelado.

Qual a diferença entre `ErrUnavailable` e `ErrInvariantViolation`?

O primeiro é transitório: vale tentar de novo. O segundo é permanente: o banco recusou algo que o domínio não deveria ter produzido.

O que `fx.As` faz?

Registra o resultado de um construtor como uma interface, para que os consumidores peçam a interface e não o tipo concreto.

Ordem de leitura do código

1. `internal/application/ports.go`
2. `internal/infra/postgres/tx.go`
3. `wallet_repository.go`
4. `errors.go` e `errors_test.go`
5. `wager_transaction_repository.go`
6. `repositories_test.go`: comece por `applyBet`, que é o caminho de escrita completo de uma aposta